

ELDERCARE INSIGHT: A MINI DATA WAREHOUSE AND ANALYTICS DASHBOARD

**01204465 Introduction to Data Mining, Data Analytics & Knowledge Discovery
Department of Computer Engineering, Faculty of Engineering
Kasetsart University**

Prepared for

**Assoc. Prof. Kitsana Waiyamai, Ph.D.
Instructor**

By

Group 05

**Kitsadee Thongbankao
6710503721**

**Kawinphop Junjuae
6710503739**

**Pirapat Rojbadin
6710504182**

22 August 2569

Abstract

This report documents a complete analytics pipeline for the United States skilled nursing facility industry, built from open data published by the Centers for Medicare & Medicaid Services covering 2019 to 2026. Five data sources are combined by an Extract–Clean–Transform–Integrate–Load process written in Python into a DuckDB star schema of two fact tables and six dimensions, and presented through an interactive Streamlit dashboard that answers eight business questions. Every figure quoted here was produced by the pipeline itself and can be reproduced from an empty folder with the commands in section 10; nothing was typed in by hand from a spreadsheet. Where the data is incomplete or a conclusion is weaker than it looks, section 11 says so explicitly rather than leaving the reader to discover it.

1 Business problem, stakeholders and business questions

1.1 The problem

An operator of skilled nursing facilities has to decide *where* to expand and *how much* to spend on staffing. The Centers for Medicare & Medicaid Services (CMS) publishes a great deal of data about every certified nursing home in the United States, so the information exists. The difficulty is its shape.

CMS publishes only the current state of the world. Each monthly release overwrites the last. History exists only as a series of archived ZIP files — 88 of them at the time of writing — that nobody has stitched into a single series. An operator asking “is occupancy recovering, and where” has no table to query. Worse, the one column that looks like history, Total Amount of Fines in Dollars, is a rolling three-year window, so it can *fall* on its own as old penalties age out. Reading it as a cumulative total produces a confident and wrong answer.

The project therefore builds the missing asset: a warehouse that keeps every period, so questions about change over time can be asked at all.

1.2 Stakeholders

User	Decision they own	What they need from the data
CEO / head of expansion	Which markets to enter or acquire in	State-by-state demand vs. supply vs. competitor quality
COO / regional manager	Where to intervene first	A ranked list of facilities whose ratings are falling
HR director	Staffing levels and retention spend	The link between nurse hours, turnover and quality
CFO / investor relations	Return and risk	Penalty exposure, occupancy, incentive multipliers
Compliance	Regulatory risk	Penalty trends, common deficiencies, Special Focus status

Table 1: The five internal stakeholders. The middle column is the test a chart has to pass: if a visual does not help someone make one of these decisions, it does not belong on the dashboard.

1.3 Eight business questions

The brief requires at least five. Eight were defined, partly for margin and partly because the right-hand column of Table 2 doubles as a design check: a question with no table behind it means the schema is not finished.

2 Measures

The brief requires at least three measures, each explained in terms of meaning, calculation and business use. Ten are defined in Table 3, which adds a fourth column — **additivity** — because that is what decides whether a dashboard totals correctly or silently lies.

The most common mistake in warehouse work: averaging a ratio

Take two facilities. The first has 10 beds and 5 residents (50%). The second has 200 beds and 180 residents (90%).

AVG(occupancy) gives $(50 + 90)/2 = 70\%$.

The correct SUM(residents)/SUM(beds) gives $185/210 = 88.1\%$.

This is not a textbook worry. Measured on this warehouse — every row, including the 96 flagged as suspect — the two methods disagree in every period:

#	Question	Owner	Answered from
BQ1	Which state is the most attractive to invest in, judged by beds per 1,000 elderly residents, occupancy, and competitor quality?	CEO	Fact_Facility_Monthly × Dim_Geography × Ref_State_Population
BQ2	How do ownership types differ in quality, staffing and penalties?	CEO, CFO	Both facts × Dim_Ownership
BQ3	How do nurse hours relate to ratings and fines, and where is the break-even?	HR, CFO	Fact_Facility_Monthly
BQ4	How high does nursing turnover have to get before it drags quality and occupancy down?	HR	Fact_Facility_Monthly
BQ5	What happened between 2019 and 2026 — has the sector recovered from COVID?	CEO, CFO	Fact_Facility_Monthly × Dim_Date
BQ6	Which facilities are deteriorating and likely to be penalised?	COO, compliance	Both facts × Dim_Facility (SCD2)
BQ7	Where and when are penalties concentrated? Which states enforce hardest per bed?	Compliance	Fact_Penalty_Event × Dim_Geography × Dim_Date
BQ8	Do large chains actually perform better, or do they just cut labour cost?	CEO	Fact_Facility_Monthly × Dim_Chain

Table 2: All eight questions have a table behind them, which is the evidence that the star schema in section 5 was designed against what has to be answered rather than drawn in the abstract.

Measure	How it is calculated	Additivity	Meaning and use
M1 Occupancy rate	SUM(avg_residents_per_day) / SUM(certified_beds)	Non-additive (ratio)	The main revenue driver; below 80% usually means a loss
M2 Resident-days	avg_residents_per_day × days in period	Fully additive	Volume of care actually sold; the weight for M1, M3, M4 and the denominator of M9
M3 Nurse hours per resident-day	reported_total_nurse_hprd, weighted by M2	Non-additive	Proxy for labour cost, and the strongest single predictor of quality
M4 Nursing turnover	total_nursing_turnover_pct, weighted by M2	Non-additive	Hidden recruiting and training cost; an early quality warning
M5 Fine value	SUM(fine_amount_usd)	Fully additive	Regulatory risk expressed in money
M6 Penalty count	COUNT(*) on Fact_Penalty_Event	Fully additive	Read with M5, separates “fined often and lightly” from “fined once and hard”
M7 Average star rating	AVG(overall_rating), 1–5	Non-additive (ordinal)	The number families actually choose on
M8 Deficiencies	SUM(cycle1_total_deficiencies)	Fully additive	Operational signal that leads the fine
M9 Fines per resident-day	M5 / M2	Computed at query time	Makes penalty exposure comparable across facilities of different size
M10 Beds per 1,000 aged 65+	SUM(certified_beds) / (pop_65plus / 1000)	Computed at query time	Market saturation; a low value means unmet demand (BQ1)

Table 3: M1–M8 are stored in the fact tables; M9 and M10 are derived at query time. No ratio is stored as a column anywhere, deliberately.

Period	SUM/SUM (correct)	AVG of ratios (wrong)	Gap
2019-01	78.98%	80.69%	+1.72 pts
2026-06	79.52%	80.65%	+1.12 pts
2026-07	80.24%	81.25%	+1.00 pts

Both numbers look like an occupancy rate, and the wrong one is wrong by enough to reorder a state ranking. The defence is structural: the fact tables store **numerator and denominator separately** and never the ratio, because storing a ratio is an invitation for somebody to average it. The view `v_check_ratio_additivity` recomputes this comparison on every build, and the dashboard shows it on its data-quality tab.

3 Data sources

The brief requires at least three sources, at least one of them more complex than a flat CSV. This project uses five, three of which are retrieved programmatically over HTTP.

#	Source	Format	What it contributes
S1	CMS Nursing Homes archived snapshots	88 multi-file ZIP archives, 20 CSVs each	The backbone. Every period of provider and penalty data; the only way to get history at all
S2	CMS Provider Data Catalog REST API	JSON over HTTP	Snapshot discovery (which periods exist, their size and date) and post-load validation of the newest period
S3	SNF Value-Based Purchasing performance	CSV per fiscal year, different shape	Incentive payment multipliers (extracted, not yet modelled)
S4	U.S. Census population aged 65+	Nested JSON over an authenticated API, with a static CSV fallback	The demand side of BQ1; the denominator of M10
S5	CMS lookup tables	CSV	Deficiency code descriptions, state averages

Table 4: Five sources. S1 is multi-period and multi-file, S2 and S4 are APIs, and S4 returns nested JSON that has to be assembled from twelve separate age-band variables before it means anything.

3.1 What is actually consumed

Only 6 of the 20 CSVs in each snapshot are used, and the counts below come from opening the June 2026 files rather than from documentation. `NH_ProviderInfo` (14,695 rows × 99 columns) feeds Fact 1 and every dimension; `NH_Penalties` (16,180 × 14) feeds Fact 2. The other four supply lookups and reserved inputs: `NH_SurveySummary` (43,952 × 41, held for a third fact), `NH_CitationDescriptions` (643 × 5), `NH_StateUSAverages` (54 × 51) and `SNF_VBP_Facility_Performance` (13,900 × 49). `NH_HealthCitations` is 165 MB and is deliberately never unpacked.

3.2 Why the source layer is harder than it looks

Three traps found by opening the real files, not by reading documentation

File names follow at least three different conventions. The 2019 archive contains `ProviderInfo_Download.csv` — no `NH_` prefix and no month. A normal 2026 archive contains `NH_ProviderInfo_Jun2026.csv`. Some 2026 archives add a dataset id on top: `4pq5-n9py_2026-07-29_NH_ProviderInfo_Jul2026.csv`. A plain glob matches some periods and silently misses others, so

the loader matches on per-era regular expressions held in one place, `config.FILE_PATTERNS`.

Some snapshots are partial. The 2026-08-06 archive is 3.5 MB and holds only 4 files, with **no penalties file at all**. Code that assumes six files per period either crashes or, far worse, loads zero penalty rows without complaint. Each period's file list is therefore checked before processing, and a missing file skips only the fact that depends on it, not the whole period.

The column structure changes across eras — 78 columns in 2019 against 99 in 2026, with wholesale renaming. This is rule Q7 in the next section.

4 Data quality problems and how they were handled

The brief requires at least three categories of data quality problem. Eight were specified up front as rules Q1–Q8, implemented as functions, and each one writes what it did to a run-log table. The counts below are the live contents of `etl_run_log` after the current build: **701 log rows** across 17 rule types.

#	Problem	What was actually found	Handling
Q1	Mismatched keys	CCN is a 6-digit code <i>with leading zeros</i> (015009). Read as a number it becomes 15009 and joins fail with no error — just missing rows. ZIP codes behave the same way.	Every column is read as <code>dtype=str</code> first; only genuinely numeric columns are converted afterwards. 626,863 values normalised.
Q2	Duplicate records	Penalty files are a rolling 3-year window, so one fine can appear in up to 36 snapshots. Fine ID only exists from period 202606 onward and is empty on all 2,470 payment-denial rows.	Two keying regimes by era: Fine ID where present (13,710 rows), otherwise a natural key plus a within-group sequence number (11,259 rows). A <code>fine_id_source</code> column records which was used.
Q3	Nulls and footnote codes	Suppressed values are published as footnote codes, not blanks, so they average as if they were data.	Suppressed rows are excluded from denominators rather than treated as zero. 1,207,433 values handled.
Q4	Impossible values	Nursing turnover above 100% in 128 facilities , the highest being 1414% — certainly an error. Occupancy above 100% also occurs, but is genuine (overflow beds).	Flag, do not delete: 384 rows marked, and 96 rows (0.21% of Fact 1) carry <code>is_suspect</code> . The dashboard has a switch to include or exclude them.
Q5	Inconsistent categories	Facility names are capitalised inconsistently in the 2019 era; left alone, roughly 17% of facilities would look like they had been renamed. Ownership has 13 raw types.	Names upper-cased; 13 ownership types collapsed to 3 groups. Between 2019 and 2026 about half of facilities changed <i>type</i> but only about 10% changed <i>group</i> , which is the empirical justification for the collapse.
Q6	Date formats	Both ends of the range use YYYY-MM-DD; no divergence found so far.	The multi-format parser is kept anyway, because roughly 28 middle periods have not been opened yet. Ambiguity is logged, never guessed.
Q7	Schema drift across years	The largest problem of the eight: 78 columns in 2019 vs. 99 in 2026, with system-wide renaming.	A per-era column mapping table in <code>columns.py</code> , sourced from Table 16 of the CMS data dictionary. 4,463 columns mapped or back-filled across 96 log entries.
Q8	Inconsistent units	June 2026 stores turnover as a percentage (median 45.30, range 4–100); the 2019 era has no such column, so the eras cannot be compared directly.	Units normalised where comparable, 107,672 values touched; BQ4 is declared cross-sectional rather than longitudinal as a result.

Table 5: The eight data quality rules, the evidence behind each, and what the pipeline does about it. Row counts come from the live `etl_run_log`.

Two policies that shaped the whole pipeline

Flag, do not delete. A row that fails a quality rule is marked `is_suspect` and kept. Deleting it would hide the imperfection from the person who most needs to see it, and some outliers are real — occupancy of 105% happens because of overflow beds. The dashboard exposes the switch, so the reader chooses.

Every rule writes to a table, not to the console. All eight rules append to `etl_run_log` in the warehouse (and to a CSV copy). The dashboard's data-quality tab reads that table live, so what the report claims about data quality and what the dashboard shows cannot drift apart.

5 Data warehouse design

The model follows Kimball's four steps. The business process is the monthly CMS publication cycle; the grain is declared before anything else; dimensions and facts follow from it.

5.1 Grain — the most important two sentences in the design

Table	One row means	Type
Fact_Facility_Monthly	One facility (one CCN) as it stood on one CMS processing date	Periodic snapshot — a row exists every period even when nothing changed
Fact_Penalty_Event	One penalty (a fine or a payment denial) against one facility on one date	Transaction — no event, no row

Table 6: Two fact tables, two grains. Having two earns the bonus point the brief offers, but the reason for two is the grain, not the bonus.

Why these two facts must not be merged

They answer at different grains, and joining them row by row multiplies the snapshot side by the number of penalties. Measured on this warehouse: a direct join inflates for-profit certified beds from **1,203,047** to **2,586,212**, a factor of **2.1**, with no error raised. Every cross-fact measure therefore aggregates each side down to a common level *first* and joins the totals — see `v_fines_per_resident_day`, which reduces both sides to (state, year) before a `FULL JOIN`.

5.2 Star schema

Fact 1 is keyed on `ccn`, not on `facility_key`

`facility_key` is an SCD2 surrogate, so one facility owns several values of it. A primary key built on it would legitimately accept two rows for the same facility in the same period, as long as a version boundary fell between them — which is exactly the duplicate the grain forbids. `ccn` is the identity that survives versioning, so it is kept in the fact table as a degenerate dimension and used in the key.

5.3 Slowly changing dimension, type 2

`Dim_Facility` tracks history with `effective_date` / `expiry_date` / `is_current`. **15,983** facilities are represented by **24,779** versions, of which 15,982 are current. Because `facility_key` is not one-to-one with `ccn`, every join from a fact to this dimension must select the version whose validity window contains the fact's date. That rule lives in exactly one function, `facts.lookup_facility_key()`, and is never rewritten inline.

Two refinements were needed once the rule met real data. Version comparison ignores columns un-

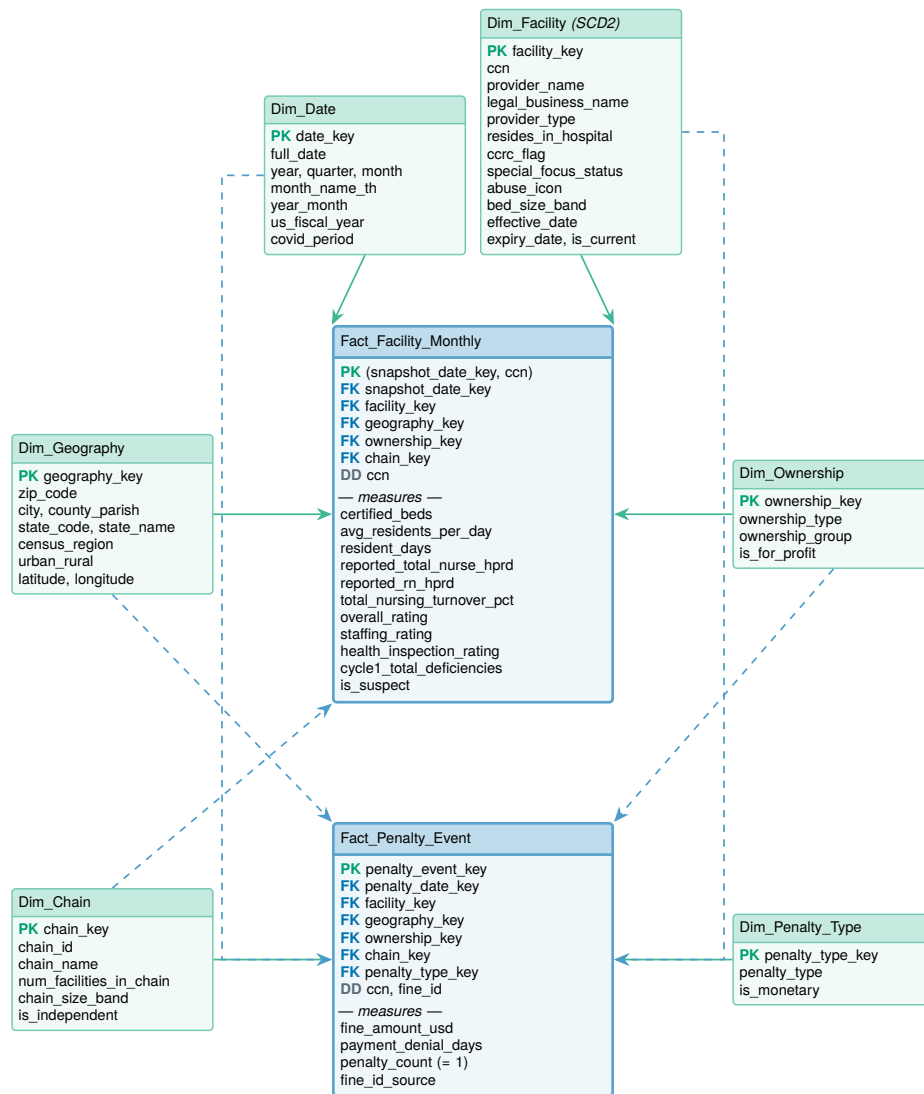


Figure 1: The star schema: two fact tables, six dimensions. Solid green edges are a fact’s own relationships; dashed blue edges mark conformed dimensions shared by both facts. **PK** primary key, **FK** foreign key, **DD** degenerate dimension — the source system’s own identifier, kept in the fact table itself. Stacking the facts vertically keeps the diagram on one portrait page and makes the shared dimensions obvious: they are the ones with edges going both up and down.

known on either side, because going from unknown to known (a column such as `abuse_icon`, absent in 2019) is late data, not a change of state — without this, crossing the 2019→2026 boundary would have opened a spurious version for all 14,300 facilities at once. And a fact older than the earliest known version attaches to that version rather than to Unknown, since penalties reach back further than the oldest snapshot loaded.

5.4 What was actually built

Table 7 is the live content of the warehouse, read out of `eldercare.duckdb` while this report was written.

Every dimension carries a `key = -1` “Unknown” row so that no fact ever holds a null foreign key — a null foreign key drops rows silently on join. `Dim_Chain` additionally distinguishes Independent (no chain) (`chain_key = 0`) from Unknown (`-1`), because a blank in 2026 means “confirmed not in a chain” whereas 2019 had no chain column at all.

5.5 The semantic layer

Ten views in `views.sql` define M1–M10 once. **Dashboards query the views, not the fact tables.** The reason is not tidiness: three of the ten measures give a plausible-looking wrong answer if computed

Table	Rows	Notes
Dim_Date	24,838	1960-01-01 to 2027-12-31, plus date_key = -1
Dim_Facility	24,779	15,983 facilities, SCD2
Dim_Geography	9,726	(ZIP, city, state) grain, 53 states/territories
Dim_Ownership	14	13 types collapsed into 3 groups
Dim_Chain	639	637 chains, plus Independent and Unknown
Dim_Penalty_Type	3	Fine, Payment Denial, Unknown
Fact_Facility_Monthly	44,966	3 periods (15,578 / 14,695 / 14,693); 96 rows flagged
Fact_Penalty_Event	24,969	20,890 fines worth \$697.8M; 4,079 denials, 114,320 days
Ref_State_Population	255	(state, year) grain — deliberately not on Dim_Geography
etl_run_log	701	Every quality rule's output

Table 7: Live contents of eldercare.duckdb, queried read-only for this report. Eight modelled tables plus two supporting tables, and ten views.

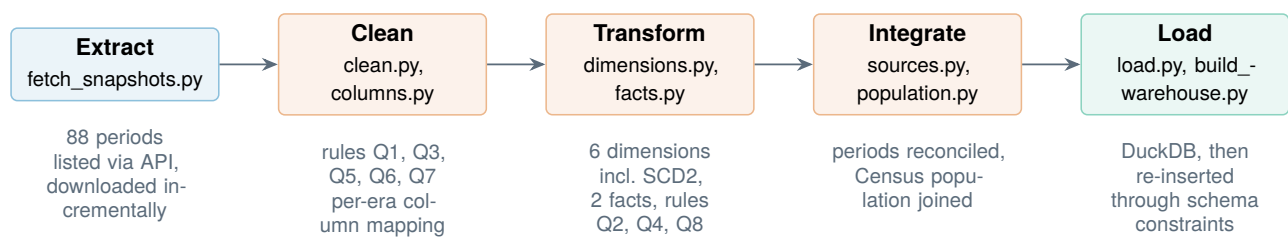


Figure 2: The five ETL stages and the module that owns each. Blue is the source stage, orange the transformation stages, green the warehouse — the same colour language as the star schema and the dashboard charts.

naively, and the wrong answer is convincing enough to survive a presentation. M1 must be a ratio of sums; M3 and M4 must be weighted by M2, or a 20-bed home outvotes a 300-bed home; M9 must not join the two facts directly.

6 The ETL process

Figure 2 shows the five stages and which module owns each.

Tools. Python 3 with pandas for all transformation, DuckDB as a single-file warehouse, Streamlit with matplotlib for the dashboard. The whole pipeline is five commands and needs no manual data editing at any step.

6.1 How each source is brought in

fetch_snapshots.py calls the CMS archive API, which returns 96 entries: 88 of type theme (real periods) and 8 of type annual_theme (yearly bundles that duplicate them). Only the former are kept. Downloads are **incremental** — a period already on disk is skipped — and only the six needed CSVs are extracted from each archive. Population comes from population.py, which prefers the Census ACS API when CENSUS_API_KEY is set and otherwise falls back to a static estimates CSV; either path lands in the same shape and the run log records which was used.

6.2 How the sources are connected

ccn is the join key throughout, which is why the leading-zero rule (Q1) is enforced at read time rather than patched later. Facts join to Dim_Facility through the SCD2 lookup described in section 5.3. Two periods, 2026-07-29 and 2026-08-06, share a processing date of 2026-07-01; without reconciliation the primary key of Fact 1 would collide immediately, so republished periods are collapsed keeping the later edition. Population is joined on state, at (state, year) grain, and deliberately not stored on Dim_Geography — that table is at (ZIP, city, state) grain, so writing a state population onto every ZIP row would let a careless SUM multiply Texas by its number of ZIP codes.

6.3 Checking before and after load

Verification results, all re-run for this report

Suite	Checks	Result
verify_dims.py	Six dimensions, including SCD2 integrity	22 / 22
verify_facts.py	Both facts, including reconciliation with CMS	22 / 22
verify_dashboard.py	Dashboard measures vs. warehouse views, to 1e-9	56 / 56
test_dashboard.py	The app run under 11 real filter combinations	11 / 11

The strongest single piece of evidence is not any of these suites, because a suite written by the same authors as the pipeline can agree with it and both be wrong. It is a reconciliation against a number CMS computed independently: restricting Fact_Penalty_Event to the fines present in the June 2026 penalties file and summing per facility reproduces the CMS Total Amount of Fines in Dollars column for **all 6,563 facilities**, totalling **\$459,054,863**, matching to the dollar.

Checking happens at three levels. **Before load**, `load.py` validates primary keys, foreign keys and SCD2 windows on the in-memory frames. **At load**, `build_warehouse.py` copies each table aside, recreates it empty from `schema.sql` with keys, uniqueness and CHECK constraints, and re-inserts the data *through* them — a bad row aborts the run and the previous state is restored, so the insert *is* the test rather than a declaration about it. This matters because the ETL writes with `CREATE OR REPLACE TABLE ... AS SELECT`, which keeps the data but discards every contract, and a Python assertion does not travel inside a `.duckdb` file handed to somebody else. The constraints were confirmed to bite: an injected `facility_key = 88888888` was rejected as a foreign key violation and `overall_rating = 9` as a CHECK violation. **After load**, the suites above query the database with SQL written independently of the code that produced it.

The pipeline is re-runnable. Dimensions are rebuilt whole every time, so no state survives between runs, and three consecutive builds produce identical results. `run_dims.py` deliberately drops the fact tables on success: surrogate keys are handed out in row order, so rebuilding dimensions from a different set of periods shifts them, and stale facts would still join — to the wrong facility, silently. A missing table is an error you notice.

7 The dashboard

The dashboard is a Streamlit application reading the warehouse read-only.

Requirement	Required	Delivered	Where
Measures summarising business performance	≥ 3	10	Header strip and tabs
Charts	≥ 5	13	All tabs
Time-based analysis	≥ 1	2	BQ5 cohort, BQ7 fines by year
Comparison of data	≥ 1	2	BQ2 ownership, BQ8 chain size
Filters / interactive controls	≥ 2	5	Period, state, ownership, chain size, suspect-row switch
Insights and recommendations	≥ 5	8	One per business question

Table 8: Compliance with the brief's dashboard requirements. `verify_dashboard.py` counts these *from the code* rather than asserting them from memory, so the table cannot quietly go out of date.

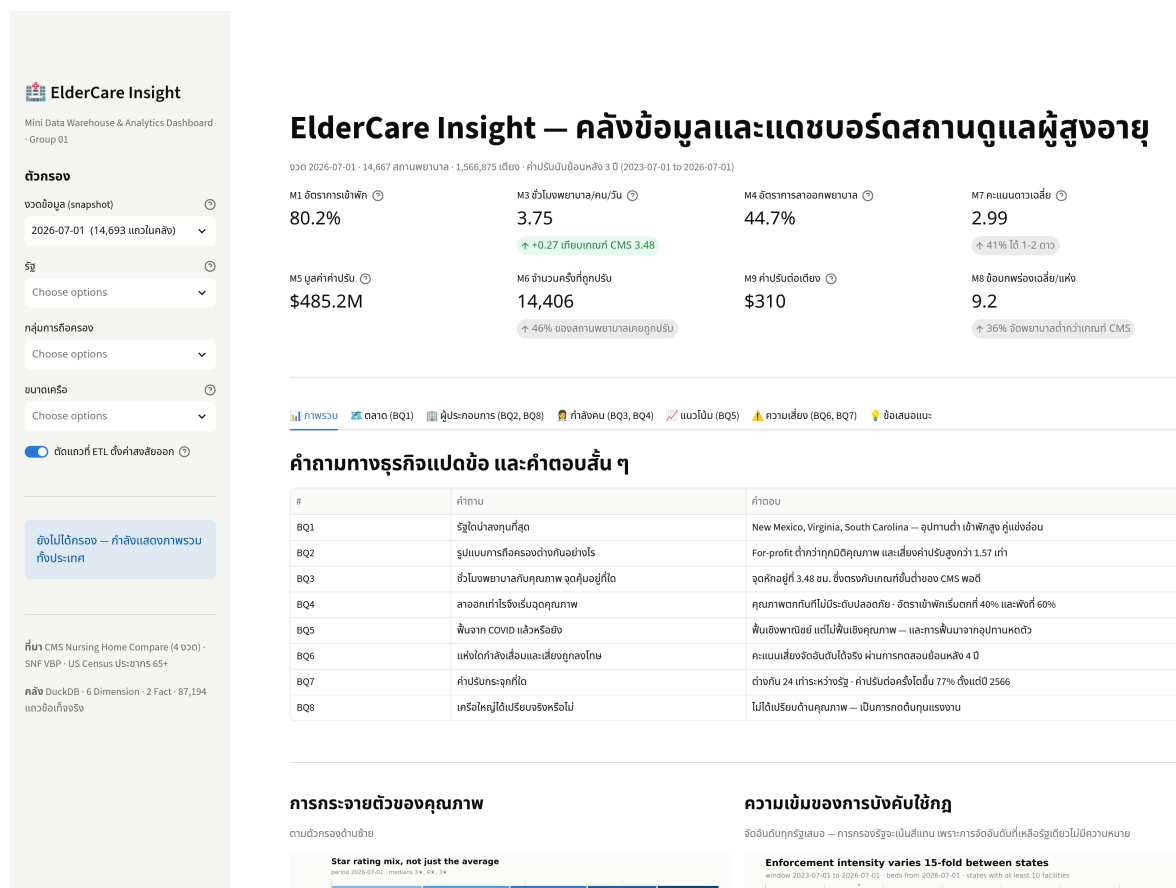


Figure 3: The complete dashboard, unfiltered, showing the national position: the measure strip across the top, and the eight business questions with their one-line answers below. Interface labels are Thai because the intended audience is Thai; the chart interiors are English. Tabs, left to right: Overview, Market (BQ1), Operators (BQ2, BQ8), Workforce (BQ3, BQ4), Trend (BQ5), Risk (BQ6, BQ7), Recommendations, Data quality.

Three design rules the dashboard follows

Read the views, never the fact tables. A dashboard that reaches past the semantic layer and writes its own AVG(residents/beds) reports occupancy about 1.7 points too high with no error. Where a filter makes a pre-aggregated view unusable, the formula is repeated in queries.MEASURES_SQL — and that duplication is *guarded*, not trusted: `verify_dashboard.py` recomputes every measure through the dashboard's own constant and asserts it matches the warehouse view to 1e-9.

Insights are computed at render time, not written into the page. A hard-coded insight is a sentence that was true of one dataset and has nothing tying it to the chart above it. Here the text and the chart read the same numbers in the same request, so they cannot contradict each other, and moving a filter recomputes the conclusion. Each insight carries an evidence-strength marker and an explicit caveat line.

Show what is missing rather than hiding it. A data-quality tab, fed live from `etl_run_log`, states the limitations of section 11 on screen rather than only in this report.

A second implementation of the same dashboard exists in the repository (`app.py`, using Plotly and reading the SQL view layer directly). Both answer BQ1–BQ8 and both pass their test suites; the screenshots in this report are of `app_mpl.py`.

8 Business insights and recommendations

All figures below are for period 2026-07-01, with the penalty window 2023-07-01 to 2026-07-01, and come from `queries_mpl.py` — the same module the dashboard calls, so the report and the screen agree

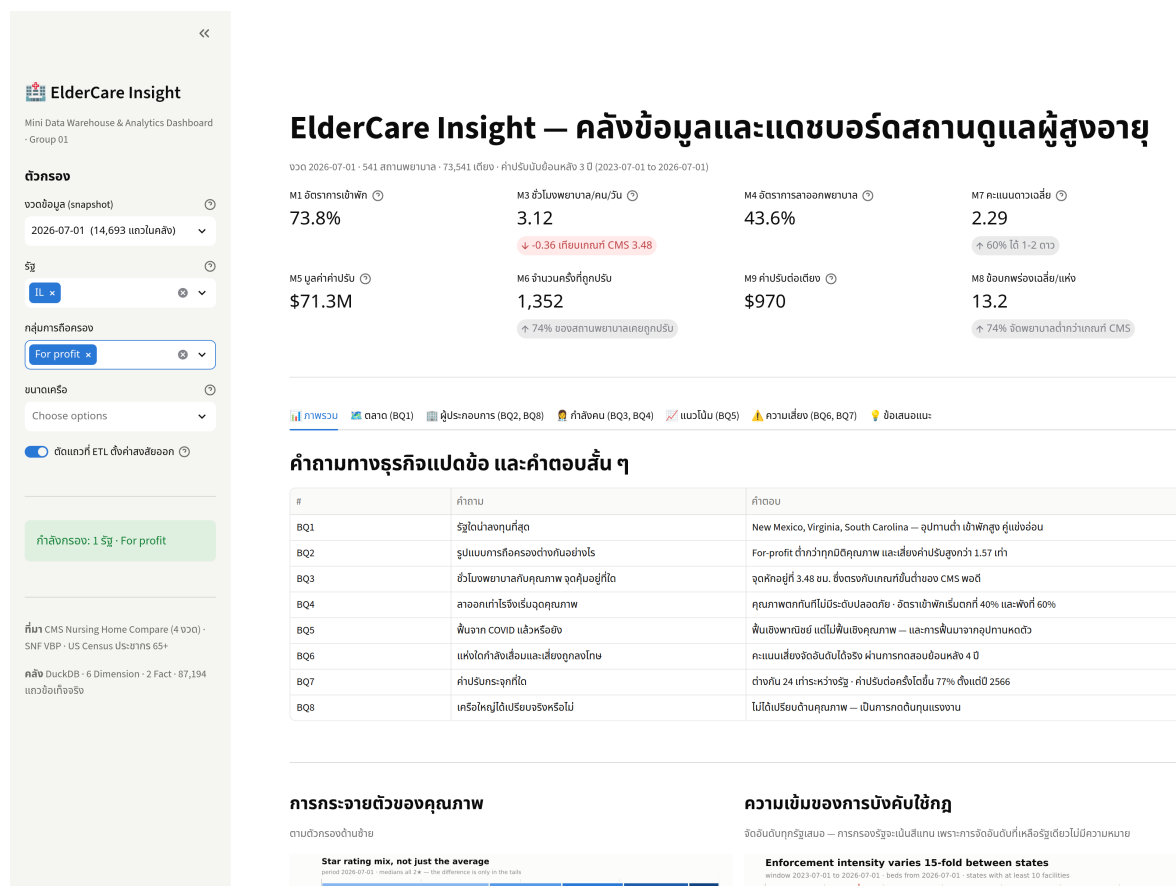


Figure 4: The same view after two filters are applied in the sidebar — state IL and ownership group For profit. Every measure and every chart recomputes; the occupancy figure in the header moves, which is how the capture is validated. `make_screenshots.py` reads a headline measure before and after applying the filter and **refuses to save the image if the value did not change**, so this figure cannot claim an interaction that never happened.

by construction rather than by coincidence.

8.1 BQ1 — where to expand

New Mexico, Virginia and South Carolina rank highest: all three combine below-average supply, above-average occupancy and weak incumbent quality. Nationally there are 25.6 beds per 1,000 people aged 65+, occupancy is 80.2% and the average competitor scores 2.99 stars; New Mexico sits at 16.1 beds, 83.4% and 2.85. **Recommendation:** shortlist these three, and avoid Arkansas, Indiana and Iowa, where more than 40 beds per 1,000 elderly coexist with occupancy in the 70s — adding capacity there only takes residents from existing homes.

8.2 BQ2 — ownership matters, and in one direction

	For profit	Non profit	Government
Facilities	10,845	2,875	947
Average star rating	2.80	3.57	3.28
Nurse hours per resident-day	3.62	4.21	4.17
Turnover	45.8%	40.4%	42.4%
Fines per bed	\$351	\$224	\$250
Ever fined	48.7%	37.1%	43.7%

For-profit facilities trail on every quality dimension and carry 1.57 times the penalty exposure per bed of non-profits. **Recommendation:** in acquisition diligence, treat for-profit ownership as a risk factor that has to be priced, not as a neutral attribute — and budget for the staffing gap, since the 0.77-star

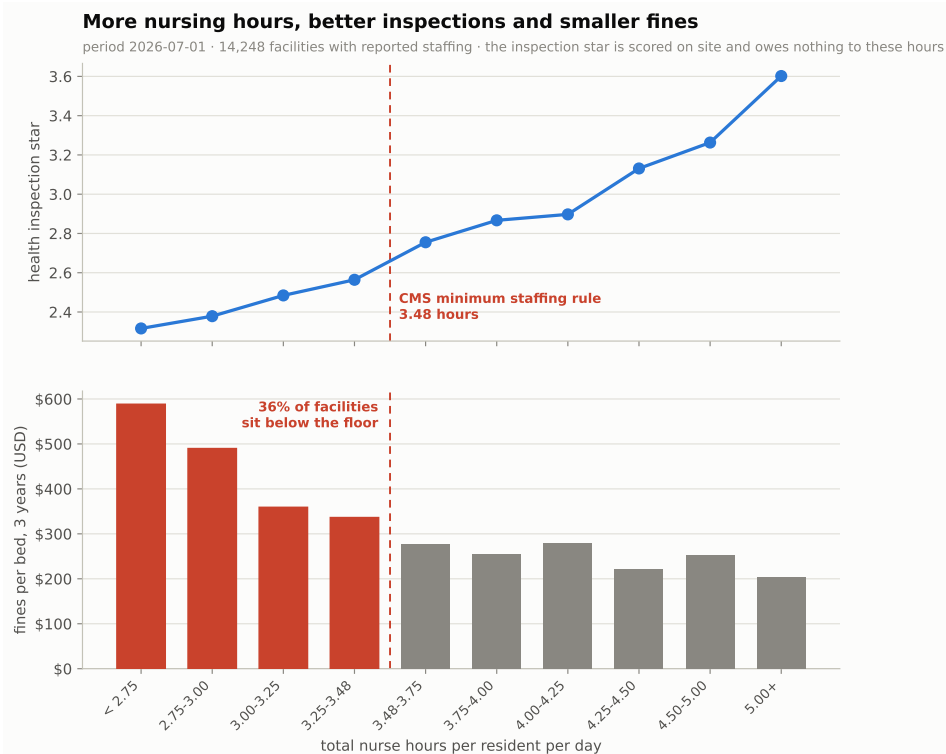


Figure 5: Health-inspection stars and fines per bed against nurse hours per resident-day. The break-even sits exactly at the CMS minimum of 3.48 hours, and the fine curve flattens above roughly 3.75.

difference tracks a 16% difference in nurse hours.

8.3 BQ3 — the staffing break-even is 3.48 hours

Below 3.48 hours, fines per bed run \$404 against \$268 above it — 1.51 times higher — and 52.6% of facilities have been fined against 39.7%. Above roughly 3.75 hours the fine curve is flat (\$289, \$264, \$283 across successive bands), so extra hours still buy stars but no longer buy risk reduction. **Recommendation:** 36% of facilities (5,134) are still below the threshold; the next staffing dollar belongs to them, not to topping up homes that already clear it.

The conclusion rests on health-inspection stars and fines rather than the overall rating, because CMS feeds nurse hours into the staffing star, which is itself a component of the overall rating — plotting hours against the overall score would be partly circular. The gradient also holds inside all three ownership groups.

8.4 BQ4 — two different thresholds for two different readers

Quality has *no* safe level of turnover: the rating falls from the very first band and keeps falling, roughly 0.2 stars for every 5 points. Occupancy behaves differently — flat near 83.5% below 40%, then losing 1.57 points immediately in the 40–45% band and collapsing to 71.7% above 65%. The national average is 44.7%. Retention is also an independent lever, not a shadow of thin staffing: even in the best-staffed quartile, moving from under 40% to over 60% turnover costs 1.34 stars — *more* than the same move costs in the thinnest-staffed quartile. **Recommendation:** HR should treat any figure above zero as worth attacking, while the CFO should defend the 40% line, because that is where the money is; retention programmes are cheaper than permanent headcount and the evidence says they work independently.

8.5 BQ5 — a commercial recovery, not a clinical one

Measured on the same 14,231 facilities open in both periods, occupancy rose 79.8% → 80.2% while the average rating fell 3.36 → 2.97 and deficiencies rose 23.7%. The recovery is also not what it appears: **1,282 facilities (8.25%) closed, taking 98,725 beds with them**, and the ones that closed had 64.9%

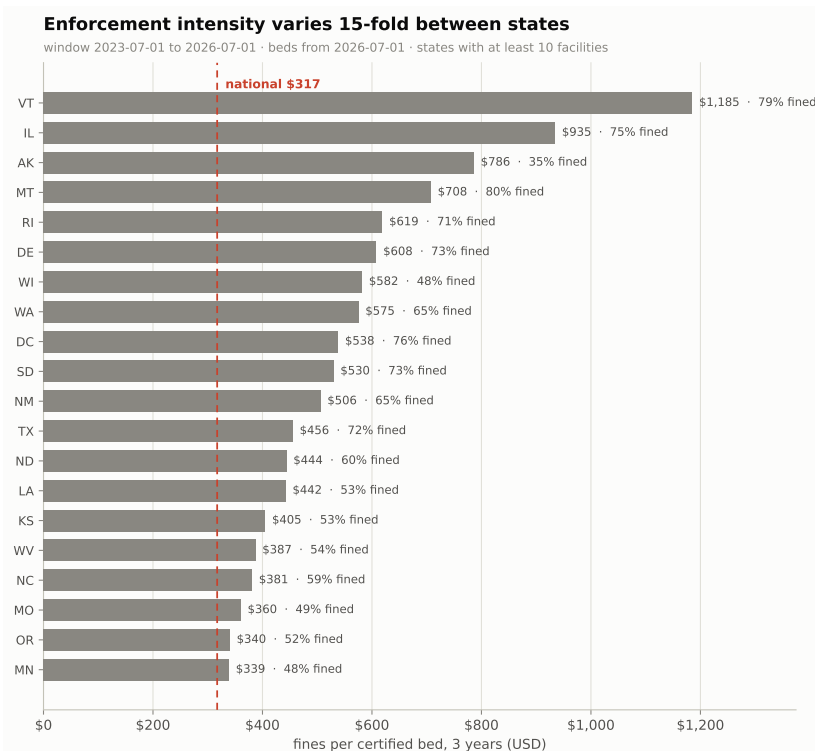


Figure 6: Fines per certified bed by state over the three-year window.

occupancy against 79.8% for survivors. Nationally beds fell 5.1% while residents fell 3.5% — **the denominator left the market, so the ratio improved. Recommendation:** do not read recovering occupancy as returning demand. Capacity decisions should be made on state-level demand, not the national ratio.

8.6 BQ6 — a risk score that survives a four-year backtest

A five-component score (Special Focus status, 1–2 stars, prior heavy fines, high deficiencies, staffing below the CMS minimum) assigned from the January 2019 data predicts outcomes more than four years later. It rises monotonically on every outcome column: facilities scoring 0 were fined 35.6% of the time and averaged \$215 per bed; those scoring 6 were fined 59.5% of the time at \$550 per bed. The high-risk group carries $1.29\times$ the penalty rate and $1.52\times$ the penalty value. **The finding that matters more than the multiplier: facilities that scored 6 in 2019 still average only 2.21 stars seven years later. Recommendation:** risk is persistent, not transient — any acquisition of a flagged facility needs a turnaround budget attached, not an assumption of mean reversion. The weights are deliberately small integers and every component is shown on screen, because a COO who cannot see why a home is on the list cannot act on it.

8.7 BQ7 — a 24-fold spread under one federal rulebook

Vermont levies \$1,364 per bed and Indiana \$57 — a 24-fold difference under the same federal rules, against a national \$324. Five states (IL, TX, CA, OH, FL) account for 43.9% of all fines on 31.8% of the beds. **Illinois is the case to watch**, being both the largest exposure (\$81.7M) and the third harshest per bed (\$960). The pattern is also changing over time: between 2023 and 2025 the average fine per event rose 76.7% while the number of events fell 19.2% — from “often and small” to “rarely and hard”. **Recommendation:** weight compliance spend by state, not by facility count, and model penalty exposure as a small number of large events.

8.8 BQ8 — scale buys occupancy, not care

Independent homes average 4.09 nurse hours and 41.6% turnover; chains of 50+ average 3.53 hours and 46.2%, and score 0.32 stars lower. The obvious objection is that large chains are almost all for-

profit, so this might just be BQ2 under another name. It is not: **nurse hours fall at every step of chain size inside all three ownership groups** — for-profit 3.75→3.54, non-profit 4.38→3.47, government 4.56→3.34. What scale genuinely delivers is commercial, not clinical: occupancy is 1.6 points higher. **Recommendation:** do not pay a quality premium for scale in an acquisition; price the occupancy advantage and budget separately for the staffing deficit that comes with it.

9 Use of generative AI, and how its output was checked

AI assistance was used throughout and is logged in 05_AI_Usage_Log/AI_Usage_Log.md, which currently holds **18 entries** against the required minimum of five. Each entry records the prompt as actually typed, what the AI proposed, what was accepted, modified or rejected, and — the field the group treats as decisive — how the output was checked.

What counts as checking is fixed by group rule: running a command and comparing to a known value, opening the real data file and counting rows yourself, checking against the CMS data dictionary, or another member reading the work and explaining it back. **What does not count:** “it reads sensibly”, “the AI said it was confident”, “asked again and got the same answer”, and “it ran without errors”.

Three examples where AI output was **not** taken as given:

Rejected outright. An AI review proposed adding payment_denial_start_date to the deduplication key for Fact 2. It does discriminate better, but an attribute CMS can revise later must not sit in a key — if the source corrects a date, the same penalty becomes a new row. The key was kept narrow.

Rejected on principle. Two suggestions were refused for the same reason — that hiding a defect is worse than showing it. Excluding is_suspect rows from the measure views by default contradicts the flag-don’t-delete rule, so the views report suspect_rows alongside instead; and hiding the BQ1 tab while M10 was incomplete would mean the reader never learns of the gap, so the tab stays with a red box naming what is missing.

Accepted as a finding, but verified independently first. An AI-assisted audit reported that the pipeline was dropping a third of its penalty rows. That was not taken on trust: the group listed the extracted files for both periods, re-counted the CSV rows with pandas (16,180 and 16,166), reproduced the original total by hand ($8,789 + 16,180 + 16,166 = 41,135$), and traced the regression to a specific commit with git show. Only then was it recorded as a real defect — the one described in section 11. The same audit’s recommendation to revert the offending change was *rejected*, because the original reasoning was sound and the correct fix is narrower.

10 Source files and how to reproduce this

Google Drive folder: _____ (view access for anyone with the link)

Dashboard: run locally with the commands below, or _____

Folder	Contents
01_Raw_Data/	Download instructions; the data itself is not stored, but is always retrievable
02_ETL/	The pipeline: extract, clean, transform, integrate, load, and two verification suites
03_Data_Warehouse/	schema.sql, views.sql, build_warehouse.py; the .duckdb file is rebuilt, not stored
04_Dashboard/	Two Streamlit applications, chart layer, notebooks, figures, screenshots, tests
05_AI_Usage_Log/	18 logged entries
06_Report/	This report, the full design document, and the eight business answers in long form

Starting from an empty folder:

```
python -m pip install -r requirements.txt
cd 02_ETL
python fetch_snapshots.py --dates 2019-01-17 2026-06-24 2026-07-29 2026-08-06
python run_dims.py && python run_facts.py && python population.py
python verify_dims.py && python verify_facts.py
cd ../03_Data_Warehouse && python build_warehouse.py --report
cd ../04_Dashboard && python verify_dashboard.py
streamlit run app_mpl.py
```

11 Limitations

Stated here rather than left for the reader to find.

1. **The penalty data is incomplete.** The deduplication rule collapses 2026-07-29 and 2026-08-06, which share a processing date, by keeping the later edition — but that edition is the 3.5 MB partial archive with no penalties file, so 16,166 penalty rows go with it. Every fines-per-bed figure here is therefore slightly understated and the latest penalty in the warehouse is dated 2026-05-13, not 2026-07-01. **Comparisons between groups remain valid, because every group is missing the same two months;** absolute levels need this caveat attached. The fix is to deduplicate per file rather than per period. Note that `verify_facts.py` still reports 22/22, having no check that every extracted file was read — a passing suite is not proof of completeness.
2. **Three time points, not a time series.** Snapshots exist for 2019-01, 2026-06 and 2026-07, with a seven-year gap, so BQ5 is a matched cohort comparison between two endpoints rather than a trend: it says where the sector ended up, not when it bottomed out. The charts break the line across the gap instead of interpolating. For the same reason M9 compares across states within a year but not across years.
3. **Chain data exists only for 2026,** so BQ8 cannot distinguish “chains degrade the homes they own” from “chains buy homes that were already weak”.
4. **Turnover exists only in the 2026 era,** so BQ4 is cross-sectional.
5. **Two population paths.** `Ref_State_Population` is populated and drives the BQ1 figures quoted here. The older `Dim_Geography.pop_65plus` column remains empty, so the `v_market_saturation` view — and the Plotly dashboard that reads it — still returns null for M10. Consolidating the view onto `Ref_State_Population` is outstanding work.
6. **The VBP source is extracted but not yet modelled;** a third fact at facility × fiscal-year grain is designed but not built.

Data source and licence

Centers for Medicare & Medicaid Services, *Provider Data Catalog — Nursing Homes*, <https://data.cms.gov/provider-data/archived-data/nursing-homes>. United States government public data, free to use with attribution. Population figures from the U.S. Census Bureau.